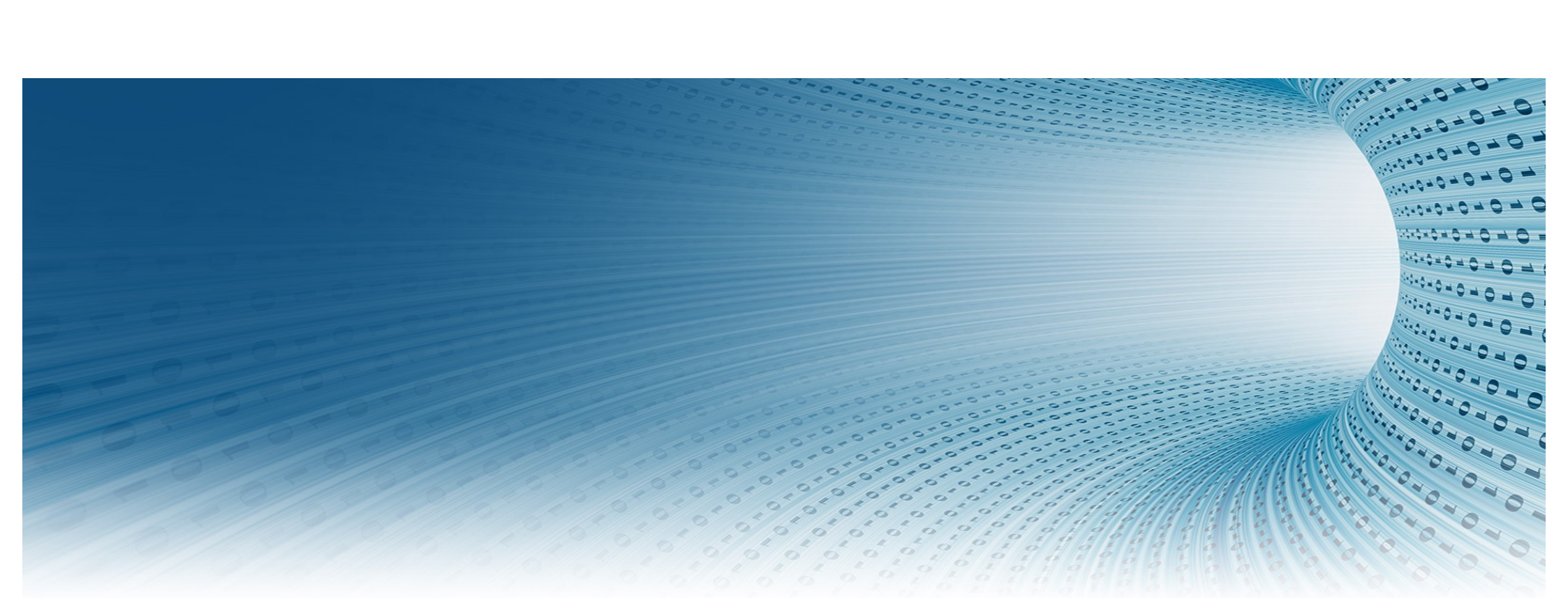


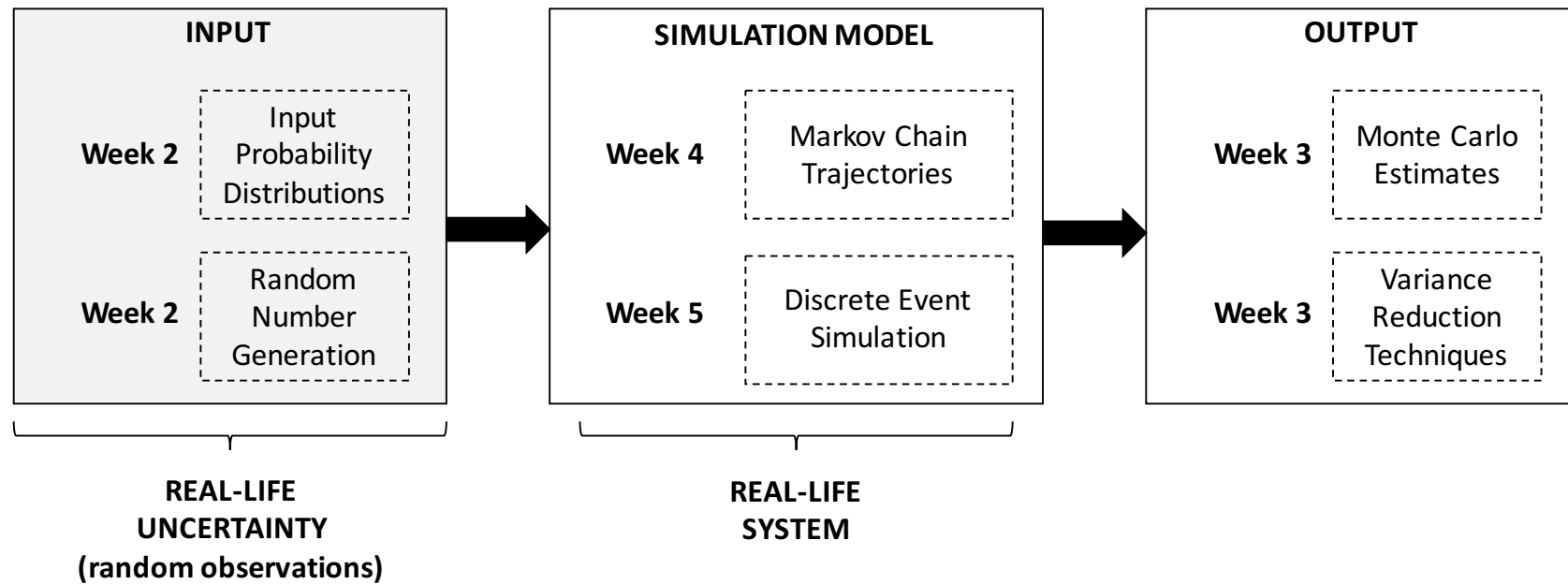


Random number generation

General Concepts and Algorithms

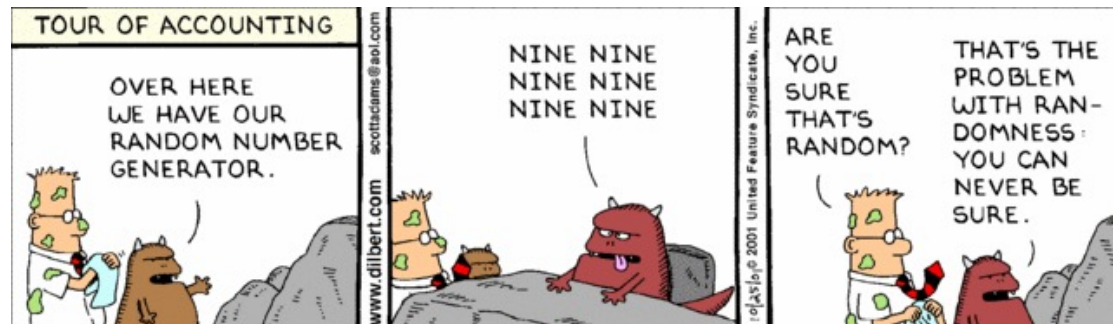
Broos Maenhout
Faculty of Economics and Business Administration
Tweekerkenstraat 2
9000 Gent

Course Schedule



Outline

- Introduction
- Uniform random number generation
- Algorithms for a URNG
- Generation of Random Observations from a Probability Distribution



Introduction

- Where do we get sequences of random numbers to carry out the simulation experiments?
 - Some physical processes are inherently random. We can construct a device that generates true random numbers, but the speed remains problematic.
 - E.g. The result of throwing a dice is random, but this is rather impractical and slow.
 - We can tabulate these random numbers, but randomness is lost once the number are tabulated.
 - We can use software algorithms to mimic random devices.
 - Advantage: Algorithms are faster than the physical process
 - Disadvantage: Algorithms are deterministic by nature.
- => Generate sequences that look like random sequences and validate these statistically

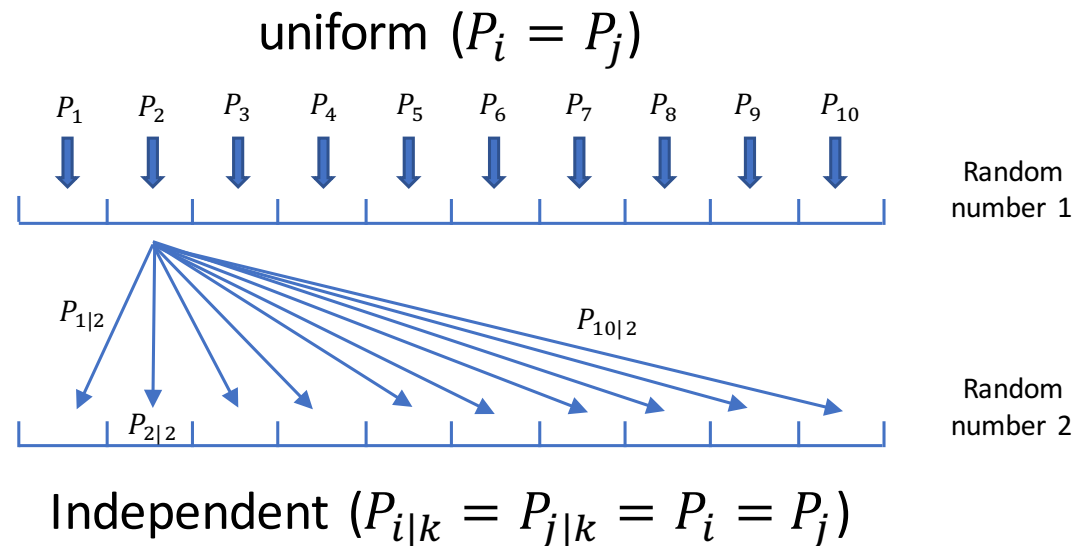
Introduction

- What?
 - Usually, the term 'random number' is reserved for random observations from a *uniform distribution*.
- How?
 - Using a Random Number Generator
 - A *random number generator* is an algorithm that produces sequences of numbers that follow a specified probability distribution and possess the appearance of randomness.
 - Devices
 - Using a physical device (spinning disc, electronic randomizer) or tables of random numbers.
 - Using computers: Excel - RAND(), Java - Math.random(), ...

Introduction

- What?
 - Usually, the term ‘random number’ is reserved for random observations from a *uniform distribution*.

(not in your slides)



Uniform random number generator

- Generic Uniform random number generator

- Definition

A uniform random number generator is a tuple $(\mathcal{S}; f; g; s)$ where $\mathcal{S}$ is the finite state space of the random number generator, $f : \mathcal{S} \rightarrow \mathcal{S}$ is the transition function, $g : \mathcal{S} \rightarrow (0, 1)$ is the output function, and $s \in \mathcal{S}$ is the initial seed. Random variables $u_0, u_1, \dots$ are then generated by executing these steps:

Step 1. Initialise the internal state: $s_0 \leftarrow s, k \leftarrow 0$.

Step 2. Calculate the next random variable: $u_k \leftarrow g(s_k)$.

Step 3. Calculate the next state: $s_{k+1} \leftarrow f(s_k)$.

Step 4. $k \leftarrow k + 1$.

Step 5. Return to step 2.

Uniform random number generator

- Generic Uniform random number generator

➤ A Bad Example

$$\mathcal{S}=\{0,1,2,3,4,5\} \quad f(x) = \text{Round}(e^x+2) \bmod 6 \quad g(x) = x/6 \quad s = 1$$

k	s_k	$u_k = g(s_k)$	$s_{k+1} = f(s_k)$
0	1	0.16666667	5
1	5	0.83333333	0
2	0	0	3
3	3	0.5	4
4	4	0.66666667	3
5	3	0.5	4
6	4	0.66666667	3

Uniform random number generator

- Some notes

- Random numbers generated by an algorithm are not actually random. They are therefore called *pseudo-random numbers*.
- Random number generators are periodic by nature, i.e. after a number of iterations the same sequence of numbers arise.
 - The size of the state space $\mathcal{S}$ is the maximal period.
 - The number of consecutive numbers in a sequence before it begins repeating itself is the *cycle length*.

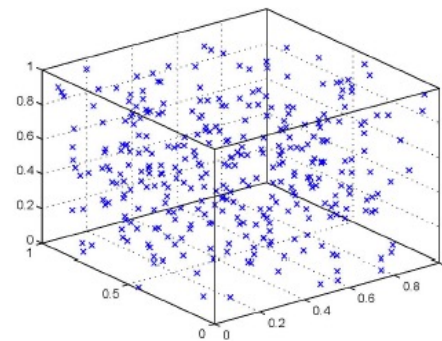
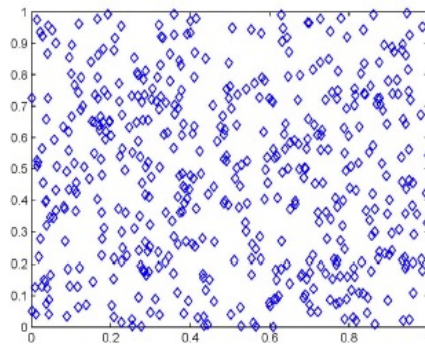
⇒ Quality differs from generator to generator, i.e. not any uniform random number generator $(\mathcal{S}; f; g; s)$ yields a sequence identical to sequences of independent uniformly distribution random variables.

Uniform random number generator

- Quality criteria for random number generators
 - Long period
 - Prevent the same sequence of numbers is generated again after some time
 - The period should be close to the state space and independent of the chosen seed
 - Computationally efficient
 - Portable between different hardware/software platforms
 - Repeatability: The same sequence can be regenerated
 - Jump-ahead: Ability to take multiple steps in the state space at once
 - Generate uncorrelated and uniformly distributed values in $]0,1[$ to avoid division by zero
- ⇒ Validate the quality of a random number generator by comparing the properties of the generated samples with the characteristics of the uniform random distribution

Uniform random number generator

- Quality assessment of a uniform random number generator
 - Check uniformity and independence via visual inspection
 - The set of tuples that can be generated by a random number generator has the form $\Psi = \{(u_0, u_1, \dots, u_{t-1}), s_0 \in \mathcal{S}\}$ where the vector $(u_0, u_1, \dots, u_{t-1})$ is deterministically generated for a given seed s_0
 - Verify the dependence of one number from its predecessor visually via
 - Plots of 2-tuples: $\{(u_i, u_{i+1})\}$
 - Plots of 3-tuples: $\{(u_i, u_{i+1}, u_{i+2})\}$



The points in Ψ should uniformly cover the unit plain or hypercube

Uniform random number generator

- Quality assessment of a uniform random number generator
 - Check uniformity and independence via visual inspection
(not in your slides)

not uniform



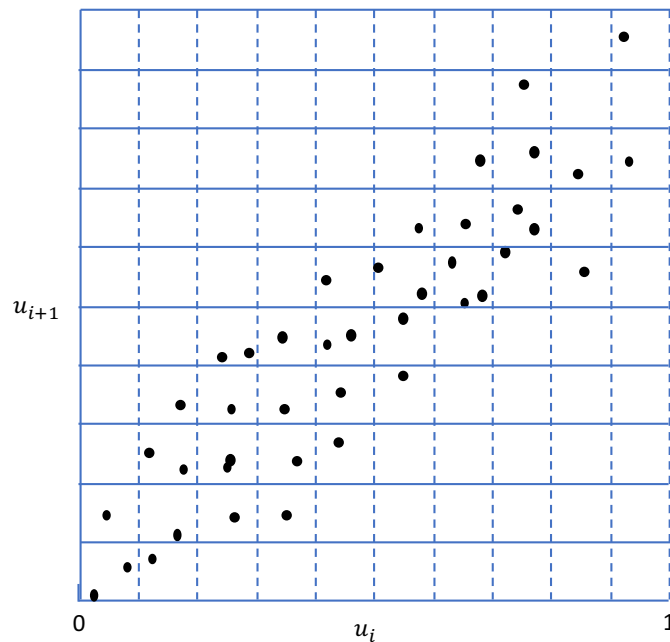
uniform



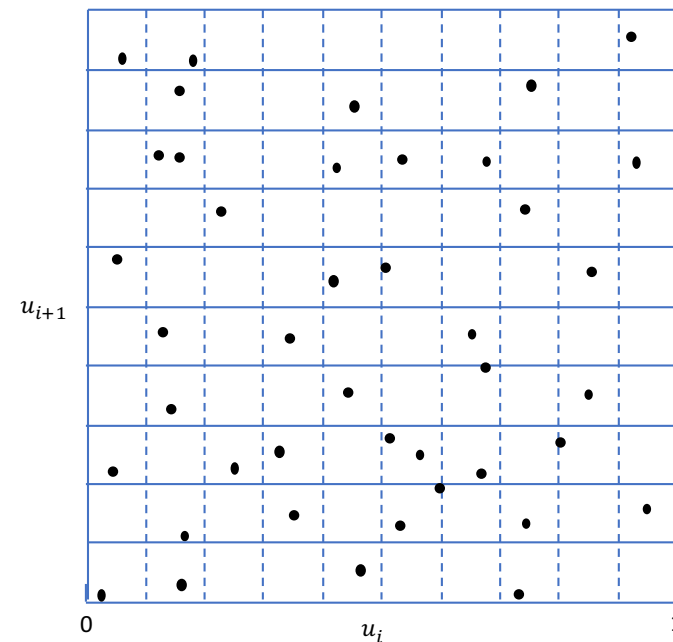
Uniform random number generator

- Quality assessment of a uniform random number generator
 - Check uniformity and independence via visual inspection

(not in your slides)



No uniform coverage
= dependency



Uniform coverage
= independency

Uniform random number generator

- Quality assessment of a uniform random number generator
 - Range and frequency test
 - Detect whether the output values fall in the prespecified range
 - Mean test
 - Verify if the mean is close to the average value 0.5 for a large number of generated values (based on the law of large numbers)
 - Variance test
 - Verify if the variance is close to the value $1/12$ for a large number of generated values

Uniform random number generator

- Quality assessment of a random number generator

- Uniformity test

- Block frequency or bucket test

- Divide the unit interval into k intervals of equal length, which cover the entire range and do not overlap

- How many intervals? When too few: Errors can go undetected

- When too many: We should have at least five samples in each interval, so we have to generate more data when there are more intervals

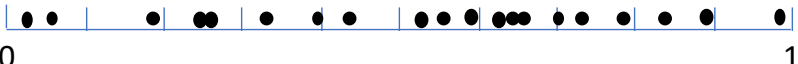
- Where to place the intervals? Determine cutoff values such that we expect an equal number of samples in each interval (= 100%/(# intervals))

- Use the χ^2 test with H_0 that the distribution is uniform

- E_i is the expected number of samples for the i th bucket and O_i is the number of samples you actually observe. You compute the following Chi-square statistic: $\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$
 - If this value is too large: the observed counts are too different from the expected counts
 - If this value is too small: the expected counts agree too well and we do not have enough randomness

Uniform random number generator

- Quality assessment of a random number generator
 - Uniformity test via block frequency or bucket test (Chi square test)
- (not in your slides)

										
Observed count	2	1	2	2	1	3	3	3	2	1
Expected count	2	2	2	2	2	2	2	2	2	2
χ^2	0	$\frac{1}{2}$	0	0	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	0	$\frac{1}{2}$

$$\sum = 3 < \chi^2_{9,0.95} = 16.919$$

=> Accept H_0

Uniform random number generator

- Quality assessment of a random number generator

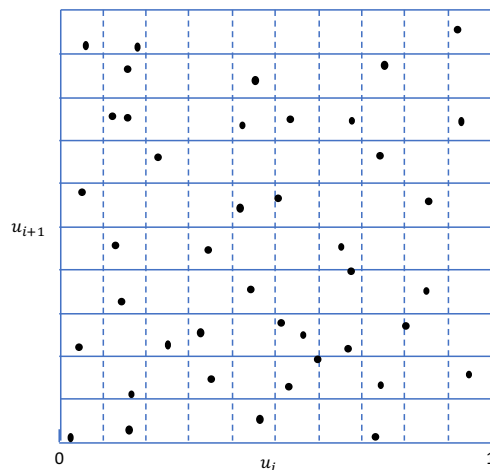
- Independence test between sequences of random numbers

- Serial test: Block frequency test

- Consider n -tuples of generated samples
- Divide the n -dimensional hypercube in subhypercubes of equal volume
- Use the χ^2 test with H_0 that the distribution is multivariate uniform

⇒ Verify correlation/independence of a sequence of generated samples

(not in your slides)



10 x 10 squares
Check if consecutive pairs
are uniformly distributed
using Chi-square test

Uniform random number generator

- Quality assessment of a random number generator

- χ^2 test explained

- $\chi = \{X_1, \dots, X_N\}$ is a sample drawn from a population with unknown distribution function $F_X(x)$
 - Null Hypothesis (H_0): $F_X(x) = F_0(x)$ for all x (with $F_0(x)$ is a completely specified distribution)
 - Alternative hypothesis (H_1): $F_X(x) \neq F_0(x)$ for some x
 - N observations grouped into k mutually exclusive categories
 - N_j : number of trial outcomes in j^{th} category
 - $N\lambda_j$: expected number for the j^{th} category
 - Test criterion: $X^2 = \sum_{j=1}^k \frac{(N_j - N\lambda_j)^2}{N\lambda_j}$ with $N = \sum_{j=1}^k N_j$

X^2 approximately χ^2 distributed with $k-1$ degrees of freedom

H_0 hypothesis accepted at level α if $X^2 < \chi_{k-1, 1-\alpha}^2$

Uniform random number generator

- Quality assessment of a random number generator

- Kolmogorov-Smirnov Test

- The bucket test is “chunky”.

(not in your slides)



- Take a large number of samples n . For each sample x_i we can compare the proportion of samples less than x_i to the proportion of samples we would expect to have seen. Compare the empirical distribution function with the theoretical distribution function:

$$\text{Empirical distribution: } F_n(x) = \frac{\text{the number of } x_i \text{ values } \leq x}{n}$$

Theoretical distribution $F(x)$ is the theoretical probability of the random number generator returning a value $\leq x$

Uniform random number generator

- Quality assessment of a random number generator

- Kolmogorov-Smirnov Test

(not in your slides)

Example

$N = 5$

Random numbers

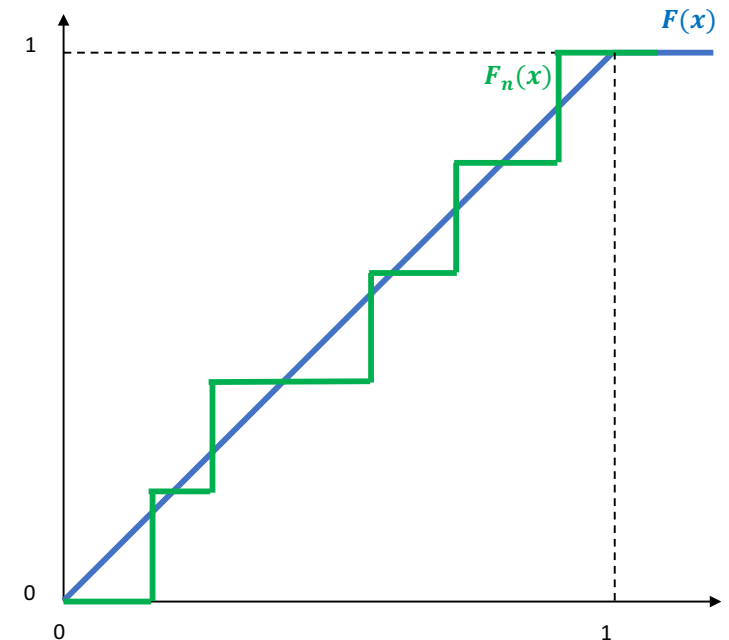
0.70, 0.27, 0.16, 0.88, 0.56

⇒ Empirical distribution $F_n(x)$

Z_i	0.16	0.27	0.56	0.70	0.88
$P(Z_i)$	0.2	0.2	0.2	0.2	0.2
$P(Z \leq Z_i)$	0.2	0.4	0.6	0.8	1

⇒ Theoretical distribution $F(x)$

Uniform distribution over interval $[0,1]$



Uniform random number generator

- Quality assessment of a random number generator

- Kolmogorov-Smirnov Test

- Compare both functions via the Kolmogorov-Smirnov test and calculate

$$K^+ = \sqrt{n} \max_{\infty} (F_n(x) - F(x)) \quad \text{the maximum value by which } F_n(x) > F(x)$$

$$K^- = \sqrt{n} \max_{\infty} (F(x) - F_n(x)) \quad \text{the maximum value by which } F_n(x) < F(x)$$

- If both distributions line up perfectly: $K^+ \approx 0$ and $K^- \approx 0$ which may be suspicious (not random)
- For large n (i.e. $n > 1000$), K^+ should lie between 0.07089 and 1.5174 around 98% of the time¹

¹Reference: Knuth, D.E., 1998, The Art of Computer Programming, Vol 2: Seminumerical algorithms, Third Edition, Addison-Wesley.

Uniform random number generator

- Quality assessment of a random number generator

➤ Kolmogorov-Smirnov Test

(not in your slides)

Example

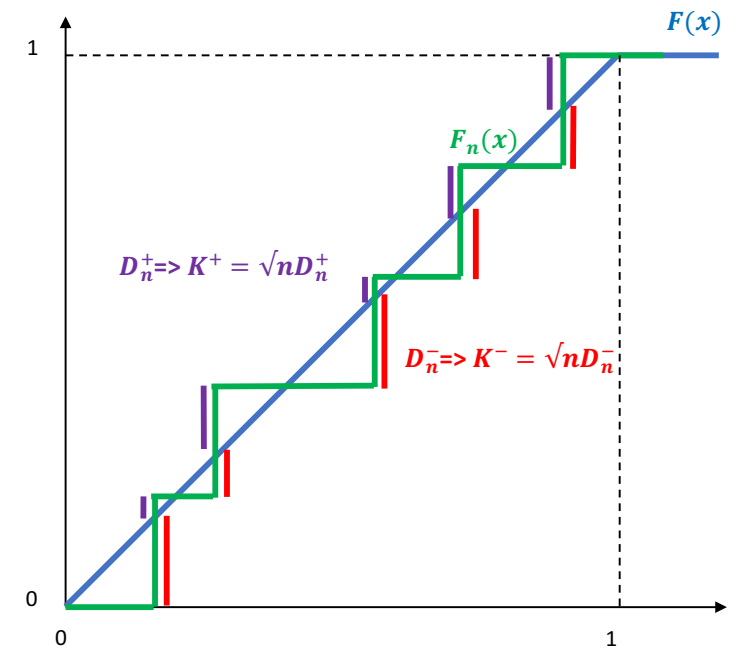
Z_i	0.16	0.27	0.56	0.70	0.88
i/N	0.2	0.4	0.6	0.8	1
$i/N - Z_i$	0.04	0.13	0.04	0.1	0.12
$Z_i - \frac{i-1}{N}$	0.16	0.07	0.16	0.1	0.08

$$\Rightarrow D_n^+ = 0.13 \text{ and } D_n^- = 0.16$$

$$\Rightarrow D_n = 0.16$$

$$\Rightarrow \text{KS test statistic: } \left(\sqrt{5} + 0.12 + \frac{0.11}{\sqrt{5}} \right) D_n = 0.385 < c_{1-\alpha} = 1.358 \quad (\alpha = 0.05)$$

Accept H_0



Uniform random number generator

- Quality assessment of a random number generator
 - General testcases and references
 - TestU01 (L'Ecuyer and Simard, 2007): L'Ecuyer and R. Simard, *TestU01: A C Library for Empirical Testing of Random Number Generators* ACM Transactions on Mathematical Software, Vol. 33, article 22, 2007.
 - DIEHARD tests (Marsaglia, 1996): G. Marsaglia, “Diehard: A Battery of Tests of Randomness,” 1996.
<http://www.stat.fsu.edu/pub/diehard/>
 - NIST SP 800-22 (National Institute of Standards and Technology, Federal US Government): A. Rukhin et al., “A statistical test suite for random and pseudorandom number generators for cryptographic applications.” Special-Pub:800-22 NIST, August 2008. [Online]. Available:
<http://csrc.nist.gov/groups/ST/toolkit/rng/documentation software.html>

Algorithms for a URNG

- Linear Congruential Generators

- Definition

A linear congruential generator (a, c, N, s) is a random number generator $(\mathcal{S}, f, g, s)$ with,

$$\mathcal{S} = \{0, 1, \dots, N-1\}, \quad f(x) = x_{i+1} = (ax_i + c) \bmod N, \quad g(x) = \frac{x_i}{N}$$

with a = the multiplier; c = the increment; N = the modulus; x_0 = the seed; $0 \leq x_i \leq N - 1$

- Theorem

A linear congruential generator (a, c, N, s) has full period, if and only if

1. c is relatively prime to N (their greatest common divisor is 1)
2. $a \bmod p = 1$ if p is a prime factor of N
3. $a \bmod 4 = 1$ if 4 is a factor of N

Algorithms for a URNG

- Linear Congruential Generators

(Exercise 1: RNG)

A linear congruential random number generator has a multiplier of 5, an increment of 3 and a modulus of 16

Questions

- What is the period of this generator?
- Using $x_0 = 3$, generate the complete cycle of random integers and their associated random numbers in the interval $[0, 1]$

Algorithms for a URNG

- Linear Congruential Generators

(Exercise 1: RNG)

➤ What is the period of this generator?

A linear congruential generator (a, c, N, s) is a random number generator $(\mathcal{S}, f, g, s)$ with,

$$\mathcal{S} = \{0, 1, \dots, N-1\}, \quad f(x) = (ax + c) \bmod N, \quad g(x) = \frac{x}{N}$$

$$\Rightarrow a = 5; c = 3; N = 16$$

A linear congruential generator (a, c, N, s) has full period, if and only if

1. c is relatively prime to N $\Rightarrow c = 3$ and $N = 16$ have no common divisors
 2. $a \bmod p = 1$ if p is a prime factor of N $\Rightarrow$ The only prime divisor of $N=16$ is 2 and $a(=5) \bmod p(=2) = 1$
 3. $a \bmod 4 = 1$ if 4 is a factor of N $\Rightarrow N = 16$ is a factor of 4; $a \bmod 4 = 1$
- $\Rightarrow$ All conditions are satisfied, hence this LCM has a full period $= N (= 16)$

Algorithms for a URNG

- Linear Congruential Generators

- Mixed congruential method

- This method generates a sequence of random integer numbers over the range from 0 to $N-1$, using the following recurrence relation:

$$x_{n+1} \equiv (ax_n + c)(\text{mod } N) \quad \text{with } a, c, N \in \mathbb{N}^+; a < N, c < N$$

- Multiplicative congruential method

- This method is a special case of the mixed congruential method where $c = 0$.

- Additive congruential method

- This method is a special case of the mixed congruential method where $a = 1$ and replaces c by some random preceding number (e.g., x_{n-1}).

- Random integer numbers are converted to uniform random numbers as follows:

$$r_n = \frac{x_n + 0.5}{N}$$

Algorithms for a URNG

- Linear Congruential Generators

- Other parameter properties

- The modulus N
 - The modulus N has to be large to approximate a uniform random variable
 - $N = 2^b$ or largest prime lower than 2^b where b is the number of bits on a computer
- The increment c
 - When $c = 0$, a full period cannot be reached and the period is at most 2^{b-2} (Knuth, 1998)
- The multiplier a
 - $a = 2^l + j$ leads to poor results
 - $a = 16807$ or $a = 630,360,016$ are commonly used

Algorithms for a URNG

- Linear Congruential Generators

- Example: Mixed congruential method

- Example Random number generation: $N=16$, $a=5$, $c=7$, $x_0=4$.

n	x_n	$5x_n + 7$	$(5x_n + 7)/16$	x_{n+1}
0	4	27	$1 + 11/16$	11
1	11	62	$3 + 14/16$	14
2	14	77	$4 + 13/16$	13
3	13	72	$4 + 8/16$	8
4	8	47	$2 + 15/16$	15
5	15	82	$5 + 2/16$	2
6	2	17	$1 + 1/16$	1
7	1	12	$12/16$	12
8	12	67	$4 + 3/16$	3
9	3	22	$1 + 6/16$	6
10	6	37	$2 + 5/16$	5
11	5	32	3	0
12	0	7	$7/16$	7
13	7	42	$2 + 10/16$	10
14	10	57	$3 + 9/16$	9
15	9	52	$3 + 4/16$	4

Algorithms for a URNG

- Congruential Methods for Random Number Generation

- Example: Mixed congruential method

- Example converting random integer numbers to uniform random numbers.

Random Integer Number	Uniform Random Number
11	0.71875
14	0.90625
13	0.84375
8	0.53125
15	0.96875
2	0.15625
1	0.09375
12	0.78125
3	0.21875
6	0.40625
5	0.34375
0	0.03125
7	0.46875
10	0.65625
9	0.59375
4	0.28125

Algorithms for a URNG

- Linear Congruential Generators

- Example: Mixed congruential method

- Example Random number generation: $N=16$, $a=4$, $c=7$, $x_0=3$

(not in your slides)

n	x_n	$5x_n + 7$	$(5x_n + 7)/16$	x_{n+1}
0	3	19	$1 + 3/16$	3
1	3	19	$1 + 3/16$	3
2	3	19	$1 + 3/16$	3
3	3	19	$1 + 3/16$	3
4	3	19	$1 + 3/16$	3
5	3	19	$1 + 3/16$	3
6	3	19	$1 + 3/16$	3
7	3	19	$1 + 3/16$	3
8	3	19	$1 + 3/16$	3
9	3	19	$1 + 3/16$	3
10	3	19	$1 + 3/16$	3
11	3	19	$1 + 3/16$	3
12	3	19	$1 + 3/16$	3
13	3	19	$1 + 3/16$	3
14	3	19	$1 + 3/16$	3
15	3	19	$1 + 3/16$	3

Algorithms for a URNG

- Congruential Methods for Random Number Generation

- A good Example

- Implementation-wise, it is most convenient to choose $N = 2^\beta$ for some β . It is then easily verified that for c odd and $a = 2^r + 1$ with $r \geq 2$, **the conditions of the theorem are** satisfied and the random number generator has full period. It is noted in the literature that good statistical results can be achieved by choosing $N = 2^{35}$, $a = 2^7 + 1$, and $c = 1$.
- Pseudocode

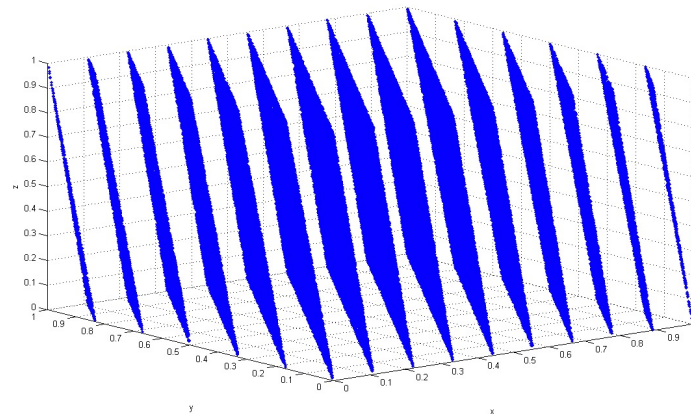
```
Initialise parameters (){  
    s = 1;  
    N = power(2, 35);  
    a = power (2, 7) + 1;  
    K = 100;                                // Number of generated random number  
}  
  
Procedure(){  
    for k in range (1,K)  
        s = (s*a+c)%N  
    return (s+0.5)/N;}
```


Algorithms for a URNG

- Congruential Methods for Random Number Generation

- A non-trivial bad example

- A bad example of a quasi random number generator that was used for a long time before its defects were discovered is the RANDU. This generator has $N = 2^{31}$, $a = 2^{16} + 3$ and $c = 0$. The initial value must be odd. Simple algebra shows that $s_{k+1} = (6 s_k - 9 s_{k-1}) \bmod 2^{31}$, or equivalently $u_{k+1} = (6 u_k - 9 u_{k-1}) \bmod 1$.
- As a result: The unit cube is badly covered by RANDU as shown in the figure below: all generated samples fall in 15 planes.



Algorithms for a URNG

- Extending/Combining random number generators

➤ Aim: get better statistical properties by combining different algorithms

- Different ways of mixing random number generators

- Method 1: Initialize $L + 1$ random number generators. Use the $L + 1^{\text{st}}$ random number generator to select which of the other generators will be used in the consecutive steps
- Method 2: Make a linear combination of different random number generators: l sequences $s_k^{(i)}$ according to the linear congruential generators $(a_i, 0, N_i, s^{(i)})$ and $u_k = \sum_{i=1}^l \frac{s_k^{(i)}}{N_i} \bmod 1$.

- Use a linear combination of preceding random integers to generate the next integer

- Example *(not in your slides)*

Given the initial seeds $s_0 = 3$, $s_1 = 5$ and $s_2 = 7$ and parameters $a_1 = 7$, $a_2 = 1$ and $a_3 = 3$ and $N = 11$

$$x_i = a_1 x_{i-1} \bmod(N) \quad \text{Cycle length} = 10 \text{ (no full period)}$$

$$x_i = (a_1 x_{i-2} + a_2 x_{i-1}) \bmod(N) \quad \text{Cycle length} = 60$$

$$x_i = (a_1 x_{i-3} + a_2 x_{i-2} + a_3 x_{i-1}) \bmod(N) \quad \text{Cycle length} = 1330$$

=> In all three cases only ten distinct integers are produced $(11-10)!$

Algorithms for a URNG

- Combining random number generators

- Example: Wichman-Hill random number generator

- Used incorrectly in Microsoft Excel 2003 and 2007 (McCullough and Heiser, 2008;)
- A decent random number generator can be obtained by combining the following three linear congruential generators:

- $L_1 = (171, 0, 30269, 1)$

- $L_2 = (172, 0, 30307, 1)$

- $L_3 = (170, 0, 30323, 1)$

$$\Rightarrow u_k = \sum_{i=1}^l \frac{s_k^{(i)}}{N_i} \bmod l$$

$\Rightarrow$ The resulting period is of the order 10^{12} .

References:

McCullough, B.D. and Heiser, D.A., 2008, On the accuracy of Statistical Procedures in Microsoft Excel 2007, Computational Statistics and Data Analysis, 52, 4570-4578

Mélard, G., 2014, On the accuracy of statistical procedures in Microsoft Excel 2010, Computational Statistics, 29, 1095-1128

Generation of Random Observations from a Probability Distribution

- Common in simulation experiments
 - Generate uniform random variables first
 - Use random variables to generate random variables with other distributions

- Different generation methods
 - Inversion method
 - Acceptance/rejection method

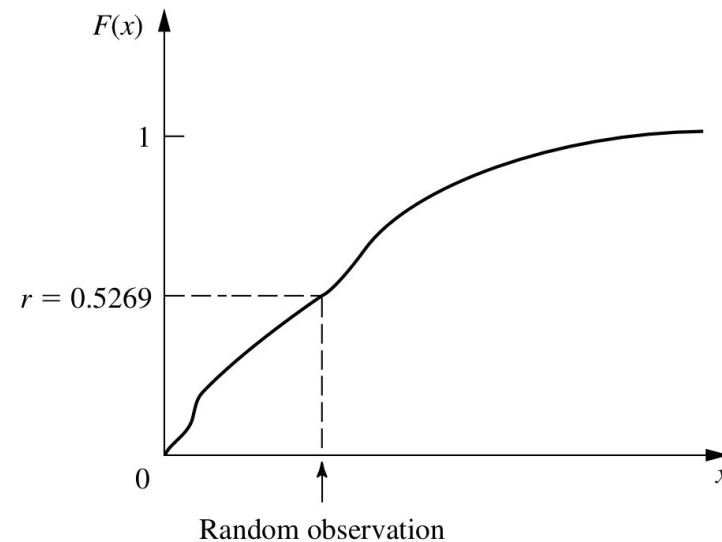
Generation of Random Observations from a Probability Distribution

- Inversion method or Inverse Transformation Method

- The cumulative distribution function: $F(x) = P\{X \leq x\}$ with X the random variable.

- Generating an observation:

1. Generate a uniform random number r between 0 and 1
2. Set $F(x) = r$ and solve for x , i.e. $F^{-1}(r)$, which is then the desired random observation from the probability distribution.



Generation of Random Observations from a Probability Distribution

- Inversion method

- Discrete random variables

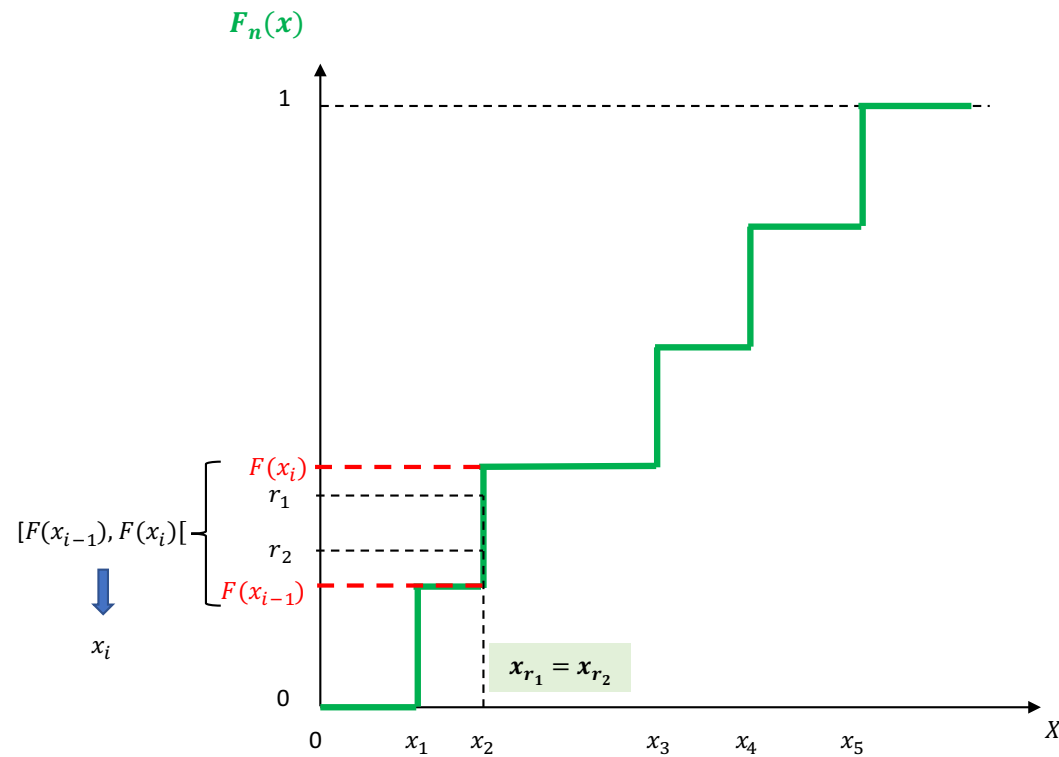
- For simple discrete distributions: allocate the possible values of a random number to the various outcomes in the probability distribution in direct proportion to the respective probabilities of those outcomes.
- In general: Let X_k be a sequence of discrete random variables taking values in the set $\{1, 2, \dots, I\}$, with $\Pr(X_k = i) = p_i$. In this case, the inversion method generates $X_k = i$ if and only if

$$\sum_{j=1}^{i-1} p_j \leq u_k < \sum_{j=1}^i p_j$$

- Execute a loop that compares u_k with $\sum_{j=1}^{i-1} p_j$ until this value exceeds u_k

Generation of Random Observations from a Probability Distribution

- Inversion method
 - Discrete random variables
- (not in your slides)



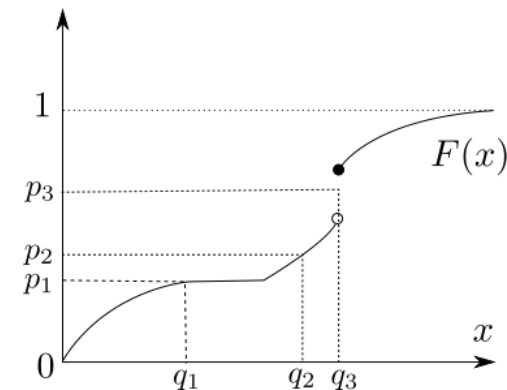
Generation of Random Observations from a Probability Distribution

- Inversion method

- Definition of Generalised inverse of a distribution function

- Cumulative distribution function $F_X(x) = \Pr[X \leq x]$
- Inverse cumulative distribution function $F_X^{-1}(u) = \inf\{x \in \mathbb{R} | F_X(x) \geq u\}$

Note: The cumulative distribution function (shown as $F(x)$) gives the p values as a function of the q values. The inverse function does the opposite: it gives the q values as a function of the p values.



- If u_k is uniformly distributed, then $X_k = F^{-1}(u_k)$ has distribution $F_X(x)$:

$$\begin{aligned} \Pr(X_k \leq x) &= \Pr(F_X^{-1}(u_k) \leq x) = \Pr(u_k \leq F_X(x)) \\ &= \Pr(u_k \in [0, F_X(x)] \leq x) = F_X(x) \end{aligned}$$

or $F_X^{-1}(u_k)$, $u_k \in [0, 1]$, is the unique real number x such that $F(x) = u_k$

Generation of Random Observations from a Probability Distribution

- Inversion method

- Example: Discrete random variables

- Suppose you timed 20 athletes running the 100-yard dash and tallied the information into four time intervals.
 1. Count the tallies and make a frequency distribution
 2. Convert the frequencies into percentages
 3. Add the frequencies into a cumulative distribution
 4. Use the percentages to develop the random number intervals

<u>Seconds</u>	<u>Tallies</u>	<u>Frequency</u>	<u>%</u>	<u>Accum. %</u>	<u>RN Intervals</u>
0-5.99		4	20	20	01-20
6-6.99		10	50	70	21-70
7-7.99		4	20	90	71-90
8 or more		2	10	100	91-00

Generation of Random Observations from a Probability Distribution

- Inversion method

- Example: Discrete random variables

(not in your slides)

Consider the following discrete random variable

x_i	1	2	3	4	5	6	7	8
$P(x_i)$	0.05	0.1	0.25	0.15	0.1	0.2	0.1	0.05
$F(x_i)$	0.05	0.15	0.4	0.55	0.65	0.85	0.95	1

u_i	0.11	0.15	0.62
-------	------	------	------

x_i	2	3	5
-------	---	---	---

If $u_i < 0.05$ then $x_i = 1$

If $0.05 \leq u_i < 0.15$ then $x_i = 2$

If $0.15 \leq u_i < 0.4$ then $x_i = 3$

etc.

Generation of Random Observations from a Probability Distribution

- Inversion method

➤ Example: Discrete random variables

(not in your slides)

Consider the following Bernoulli random variable

x_i	0	1
$P(x_i)$	0.2	0.8
$F(x_i)$	0.2	1

$\Rightarrow u_i$	0.11	0.15	0.62	0.94
x_i	0	0	1	1

If $u_i < 0.2$ then $x_i = 0$

If $0.2 \leq u_i < 1$ then $x_i = 1$

Generation of Random Observations from a Probability Distribution

- Inversion method

- Continuous random variables

- Exponential distribution:

- The cumulative distribution function: $F(x) = 1 - e^{-ax}, x \geq 0$

- Setting $F(x) = u_k$ yields $1 - e^{-ax} = u_k$ or
$$\Leftrightarrow \ln e^{-ax} = \ln(1 - u_k)$$
$$\Leftrightarrow x = -\frac{\ln(1 - u_k)}{a}$$

- As $1 - u_k$ is also uniformly distributed, $x = -\frac{1}{a} \ln u_k$
- This yields x as the desired random observation from the exponential distribution.

(not in your slides)

Generate random observations from exponential distribution with $a = 4$

$\Rightarrow u_i$	0.11	0.15	0.62	0.94
x_i	0.5518	0.4743	0.1195	0.0155

Generation of Random Observations from a Probability Distribution

- Inversion method

- Continuous random variables

- Weibull: $F_X(x) = 1 - e^{-\left(\frac{x}{a}\right)^b}$ $\Rightarrow x = a(\ln(U))^{-\frac{1}{b}}$
- Pareto: $F_X(x) = 1 - x^{-a}$ $\Rightarrow x = \frac{1}{U^{\frac{1}{a}}}$
- Logistic: $F_X(x) = (1 + e^{\frac{x-\mu}{b}})^{-1}$ $\Rightarrow x = \mu - b \ln\left(\frac{1}{U} - 1\right)$
- Uniform: $F_X(x) = \frac{x-a}{b-a}$ $\Rightarrow x = a + (b-a)u$

All these distributions are characterized by the fact that they have a significant probability of taking on very large values.

(not in your slides)

Generate random observations from uniform distribution from interval $[a,b] = [2,6]$

$\Rightarrow u_i$	0.11	0.15	0.62	0.94
x_i	2.44	2.60	4.48	5.76

$$x = a + (b - a)u$$

Generation of Random Observations from a Probability Distribution

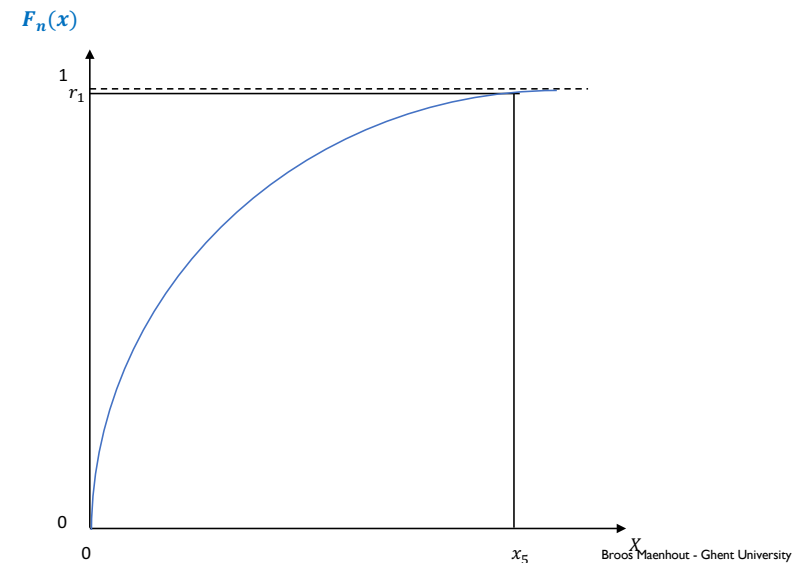
- Inversion method

- Remarks

- The inversion method only works if we have a closed-form expression for the cumulative distribution function. This is for example not possible for random variables distributed according to the normal distribution.
- Analytical methods for generating the random variables often are inaccurate for the tails of the distributions
 - extremely large values of X are generated by small values of u , quantisation effects (u is a multiple of $1/N$) may not be negligible
 - implementations of some functions (e.g. $\ln$) are possibly inaccurate for large values

⇒ Solution: Use approximations of inverse functions

or apply the acceptance-rejection method



Generation of Random Observations from a Probability Distribution

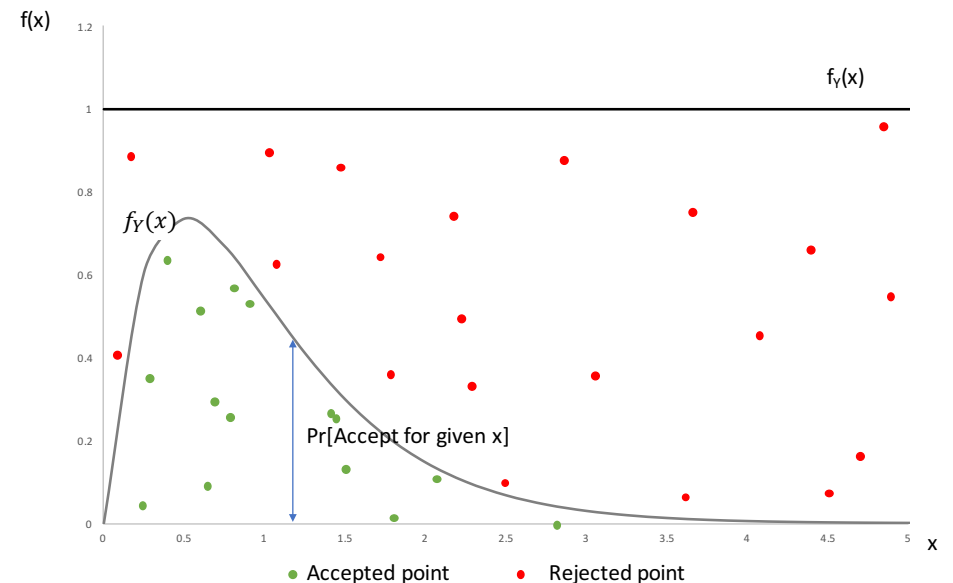
- The Acceptance-Rejection Method
 - Assume that the density function $f_Y(x)$ is known and that one can generate random variables with density $f_X(x)$ such that $Mf_X(x) \leq f_Y(x)$ for all x and a constant M
 - The acceptance-rejection algorithm operates as follows:
 1. Generate X from the density $f_X(x)$
 2. Generate U uniform in $(0,1)$
 3. If $U \leq f_Y(X)/(Mf_X(X))$, return $Y \leftarrow X$, else go back step 1.
 - Probability of acceptance a sample point $x = f_Y(Y)/(Mf_X(Y))$
 - Theorem
 - The acceptance-rejection algorithm returns random variables with density f_Y .

Generation of Random Observations from a Probability Distribution

- The Acceptance-Rejection Method

- General idea

- Sample randomly points within the box (in this case it is a rectangular box)
 - 2 random numbers are required to determine a point in the box (r_1 determines x ; r_2 calculates $f(x)$)
- Accept point when $r_2 M f_X(x) \leq f_Y(x)$: accept point
 - More samples will be accepted in the area where the probability density is high. Where $f_Y(x)$ is small for x , sample points are rejected with a higher probability
- For many different samples, we obtain a good fit with the probability density function

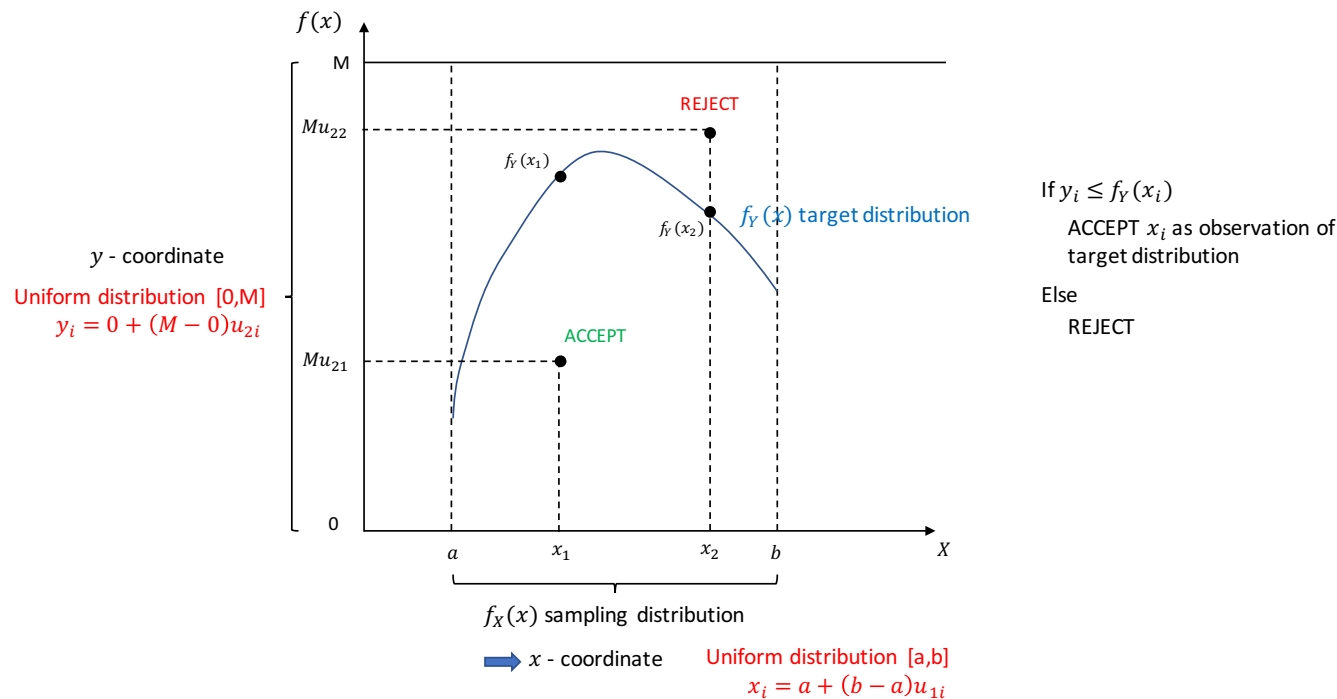


Generation of Random Observations from a Probability Distribution

- The Acceptance-Rejection Method

- General idea

- Sample randomly points within the box (in this case it is a rectangular box)
 - 2 random numbers are required to determine a point in the box (r_1 determines x ; r_2 calculates $f(x)$)

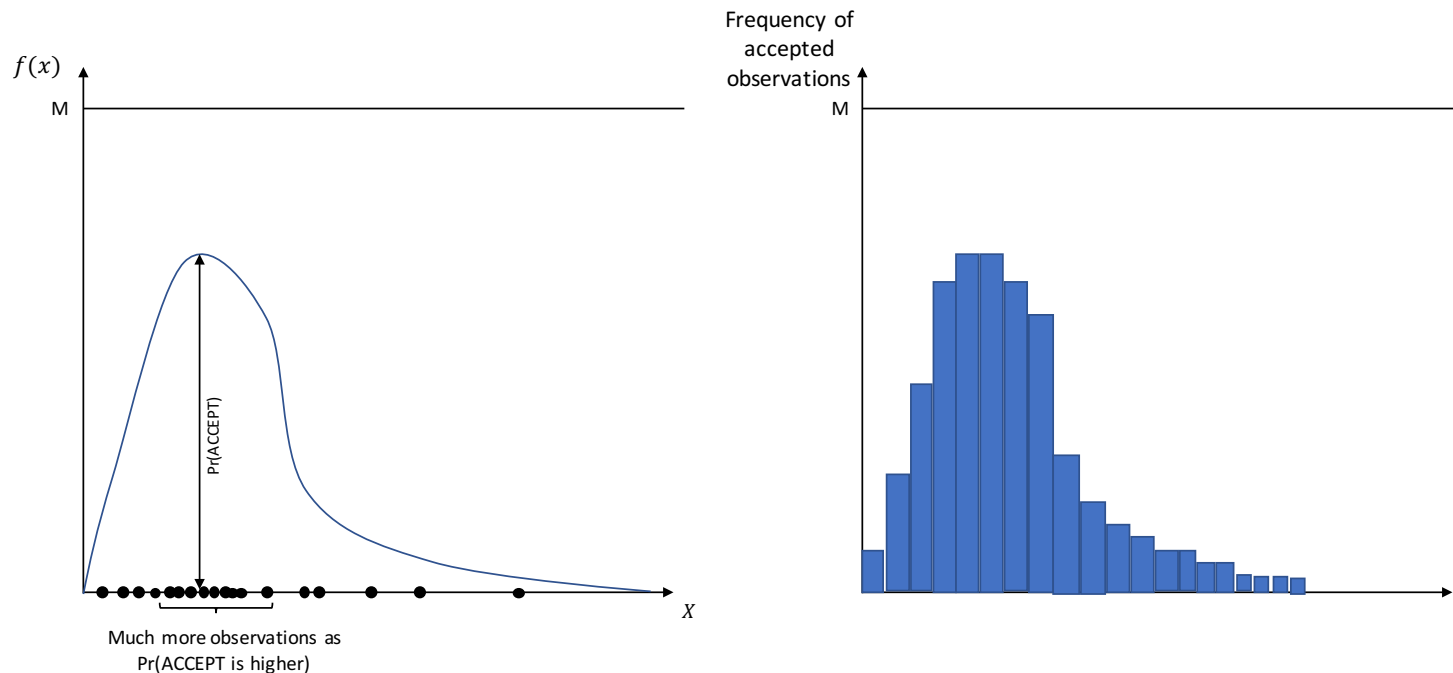


Generation of Random Observations from a Probability Distribution

- The Acceptance-Rejection Method

- General idea

- Accept point when $r_2 M f_X(x) \leq f_Y(x)$: accept point
 - More samples will be accepted in the area where the probability density is high. Where $f_Y(x)$ is small for x , sample points are rejected with a higher probability
- For many different samples, we obtain a good fit with the probability density function

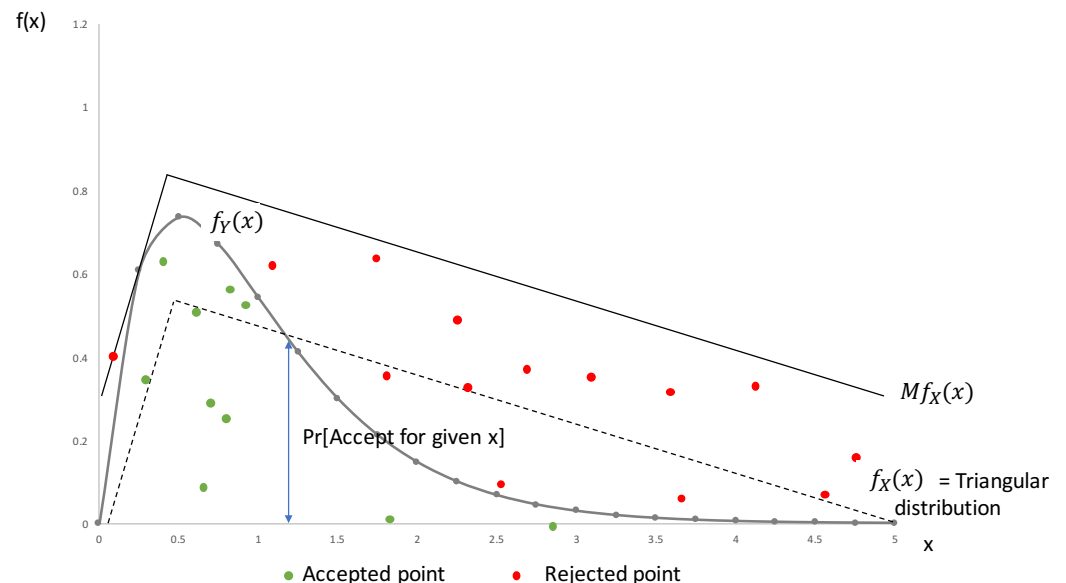
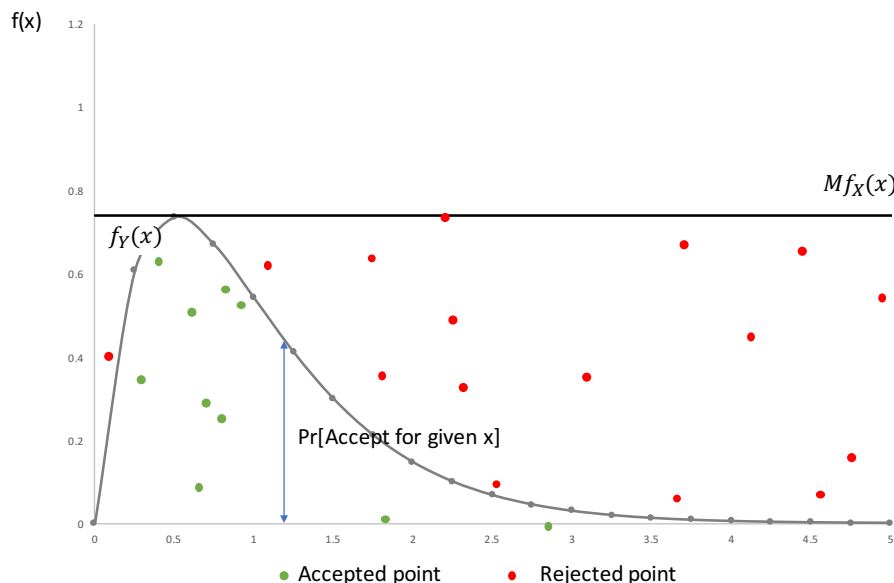


Generation of Random Observations from a Probability Distribution

- The Acceptance-Rejection Method

- General idea

- The larger the area above the curve, compared to the area below the curve the worse the efficiency of the algorithm. As a result we should limit the box as much as possible. The solution is twofold:
 - Select an accurate value for M (the maximum value) such that $f_X(x) \leq M f_Y(x)$
 - Select an accurate function $f_Y(x)$ for which it is easy to get random observations via the inversion method (the closer the box, the higher the probability of acceptance)



Generation of Random Observations from a Probability Distribution

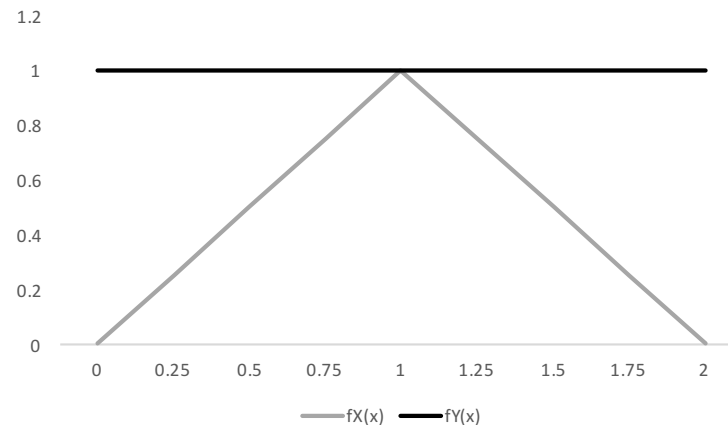
- The Acceptance-Rejection Method

- A simple example

- Given the triangular distribution with the following probability density function $f(x)$ in $[a, b]$

$$\text{e.g. } f_Y(x) = \begin{cases} x & \text{if } 0 \leq x \leq 1 \\ 1 - (x - 1) & \text{if } 1 \leq x \leq 2 \\ 0 & \text{otherwise} \end{cases}$$

Step 0. Select a constant M such that M is the largest value of $f(x)$ for any x , i.e. in this case $M=1$ over the interval $[a,b]=[0,2]$. We assume that $f_Y(x) = 1$



Generation of Random Observations from a Probability Distribution

- The Acceptance-Rejection Method

- A simple example

Step 1a. Generate a uniform random number r_1 between 0 and 1.

Step 1b. Set $x = 2 r_1$ (i.e. $x = a + r_1 \cdot (b-a)$, which ensures that each member of $[a, b]$ has an equal chance to be chosen)

Step 2. Accept x with

$$\text{Probability} = \begin{cases} x & \text{if } 0 \leq x \leq 1 \\ 1 - (x - 1) & \text{if } 1 \leq x \leq 2 \end{cases}$$

to be the desired random observation (since this probability equals $f(x)$). Otherwise reject x and return to step 1.

Implementation: Step 2a. Generate a uniform random number r_2 between 0 and 1. Set $y = r_2 \cdot M$

Step 2b. If $y \leq f(x)$, **return** x as the next random observation; Else return to step 1.

Generation of Random Observations from a Probability Distribution

- The Acceptance-Rejection Method

- A simple example

(not in your slides)

r_1	r_2	x_i	y_i	$f(x_i)$	$y_i < f(x_i)?$
0.11	0.54	0.22	0.54	0.22	REJECT
0.73	0.19	1.46	0.19	0.54	ACCEPT
0.39	0.15	0.78	0.15	0.78	ACCEPT
0.89	0.34	1.78	0.34	0.22	REJECT

Accepted observation for triangular distribution: 1.46; 0.78

Generation of Random Observations from a Probability Distribution

- The Acceptance-Rejection Method

- A second example

- Given the probability density function $f_Y(x) = \sqrt{\frac{2}{\pi}} e^{-x^2/2}$, $x \geq 0$
- Use the probability function $f_X(x) = e^{-x}$, $x \geq 0$ and set $M = \sqrt{\frac{2}{\pi}} e^{1/2} \approx 1.3155$

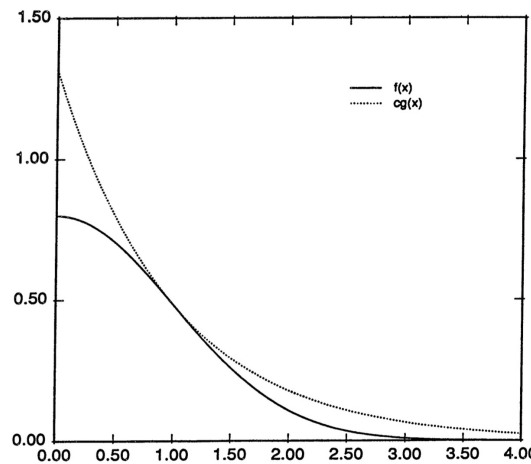


FIG. 1. Acceptance-rejection sampling from the half-normal distribution, using the exponential distribution.

Reference: Flury, B.D., 1990, Acceptance-rejection method made easy, SIAM Review, 32, 474-476.